DVWA ATTACK LAB

FINAL PROJECT REPORT

Five Authorized Exploitation Write-ups
Low Security Level | Local Training Environment

STUDENT	M. Sahan Sithumina
BOOTCAMP	DVWA Attack Lab - From Recon to Exploitation
PROVIDER	DevTown
INSTRUCTOR	Prathamesh Tikle
SUBMISSION DATE	July 2026

AUTHORIZED LAB USE ONLY

All testing described in this report was performed against a local DVWA instance created for cybersecurity education.

Project Overview

This report documents five different vulnerabilities exploited in Damn Vulnerable Web Application (DVWA) at the Low security level. The selected labs demonstrate a range of web-application weaknesses across four bootcamp days: weak authentication controls, operating-system command injection, unrestricted executable file upload, SQL query manipulation, and cross-site request forgery. All evidence was collected from the authorized local DVWA environment at http://localhost:8001/DVWA.

> Submission-ready evidence
> This final version includes original screenshots from the completed local DVWA labs. The screenshots have been cropped only to improve readability; the DVWA heading, selected lab, input/result, and other relevant evidence remain visible.

Lab Environment

Item	Configuration Used
Target application	Damn Vulnerable Web Application (DVWA)
Lab URL	http://localhost:8001/DVWA
Security level	Low
Scope	Local, intentionally vulnerable training environment only
Tools	Web browser, XAMPP local server, and text editor

Selected Lab Coverage

No.	Vulnerability	Level	Bootcamp Day
1	Brute Force	Low	Day 2
2	Command Injection	Low	Day 2
3	File Upload	Low	Day 3
4	SQL Injection	Low	Day 4
5	CSRF	Low	Day 5

LAB 1 | Brute Force

LOW
Day 2

1. Lab Name & Security Level: Brute Force - Low

2. What I Was Trying To Do: Identify the valid admin password by testing a small set of candidate passwords against a login form with no visible attempt throttling.

3. Steps I Took

1. Logged in to DVWA, opened DVWA Security, selected Low, and confirmed the setting.
2. Opened the Brute Force lab and entered the username admin.
3. Tested the candidate passwords 123456, admin, letmein, and qwerty; each attempt was rejected.
4. Submitted admin with the password password.
5. The page displayed “Welcome to the password protected area admin,” confirming that the valid credential had been found.
6. Logged out and repeated the successful credential once to verify the result before capturing the evidence screenshot.
7. Kept all testing within the local DVWA training environment.

Payload / Input Used

```
Username: admin
Password candidates: 123456, admin, letmein, qwerty, password
Successful credential: admin / password
```

4. Screenshot Evidence

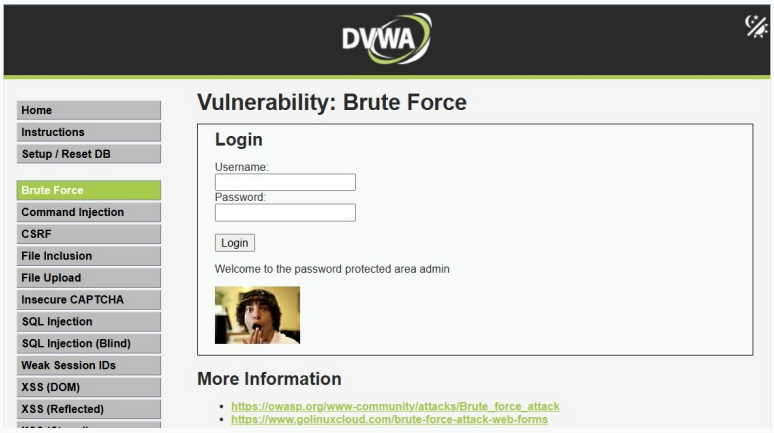

Figure 1. Brute Force - successful access to the protected admin area.

5. What This Proves
The form accepted repeated password attempts without showing lockout, delay, or CAPTCHA controls. Once the correct password was submitted, the protected area was immediately accessible.

6. How To Fix It
Apply server-side rate limiting, progressive delays or temporary lockout, multi-factor authentication, monitoring, and generic failure messages. Store passwords with a modern salted hash such as Argon2id or bcrypt.

LAB 2 | Command Injection

LOW
Day 2

1. Lab Name & Security Level: Command Injection - Low

2. What I Was Trying To Do: Make the server execute an additional Windows command through the IP-address field instead of performing only the intended ping operation.

3. Steps I Took

1. Opened the Command Injection lab with DVWA security set to Low.
2. Entered 127.0.0.1 and submitted it to record the normal ping response.
3. Replaced the normal value with 127.0.0.1 && dir and submitted the form.
4. The page returned the ping output and then displayed the contents of C:\xampp\htdocs\DVWA\vulnerabilities\exec.
5. This confirmed that the application passed the supplied input to the Windows command shell.

Payload / Input Used

```
Normal input:
127.0.0.1

Injected input:
127.0.0.1 && dir
```

4. Screenshot Evidence

Figure 2A. Normal ping using 127.0.0.1.

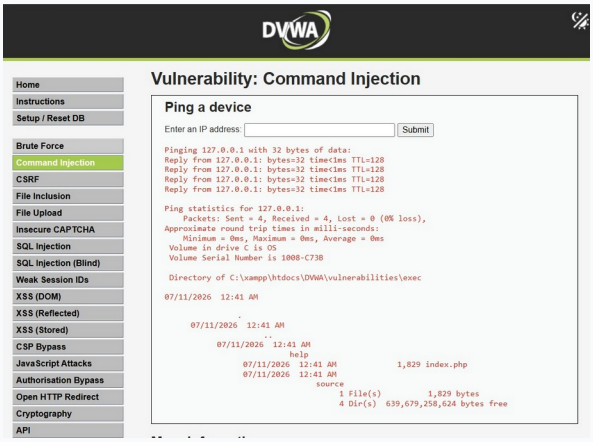

Figure 2B. Injected dir command reveals the server directory.

5. What This Proves
Untrusted input was concatenated into a shell command. The && operator caused an additional command to run with the privileges of the web-server process.

6. How To Fix It
Do not pass raw input to shell functions. Validate the value strictly as an IP address, use a networking API instead of the command shell, apply allowlists, and run the service with minimum privileges.

LAB 3 | File Upload

LOW
Day 3

1. Lab Name & Security Level: File Upload - Low

2. What I Was Trying To Do: Upload a harmless PHP proof file and confirm that DVWA stored it in a web-accessible location where the server executed it.

3. Steps I Took

1. Created a file named shell.php containing a simple PHP echo statement.
2. Opened the File Upload lab with DVWA security set to Low, selected shell.php, and clicked Upload.
3. DVWA displayed a PHP GD module warning, but it still reported ../../hackable/uploads/shell.php successfully uploaded.
4. Opened http://localhost:8001/DVWA/hackable/uploads/shell.php in the browser.
5. The browser displayed “Check the project,” confirming that the uploaded PHP file was executed by the server.

Payload / Input Used

```
<?php
echo "Check the project";
?>

Uploaded URL:
http://localhost:8001/DVWA/hackable/uploads/shell.php
```

4. Screenshot Evidence

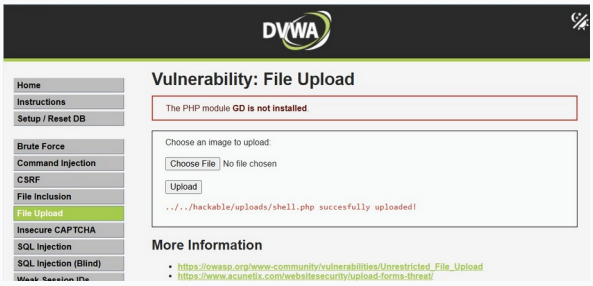

Figure 3A. DVWA confirms shell.php was uploaded.

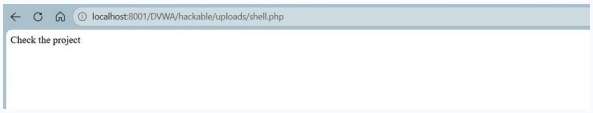

Figure 3B. Opening shell.php executes the PHP proof text.

5. What This Proves
The upload feature accepted a .php file and placed it inside a web-accessible directory where PHP execution was enabled. Even a harmless proof file demonstrates the possibility of server-side code execution.

6. How To Fix It
Allow only necessary file types using extension, MIME, and file-signature checks; rename files; store uploads outside the web root; disable script execution in upload directories; enforce size limits; and scan content.

LAB 4 | SQL Injection

LOW
Day 4

1. Lab Name & Security Level: SQL Injection - Low

2. What I Was Trying To Do: Change the User ID query so the application returned every user record instead of the single requested record.

3. Steps I Took

1. Opened the SQL Injection lab with DVWA security set to Low.
2. Entered 1 and submitted it; the normal response returned only the admin record.
3. Replaced the ID with the Boolean payload 1' OR '1'='1.
4. Submitted the payload and observed records for admin, Gordon Brown, Hack Me, Pablo Picasso, and Bob Smith.
5. Compared the normal and injected outputs to confirm that the always-true condition changed the query logic.

Payload / Input Used

```
Normal input:
1

Injected input:
1' OR '1'='1
```

4. Screenshot Evidence

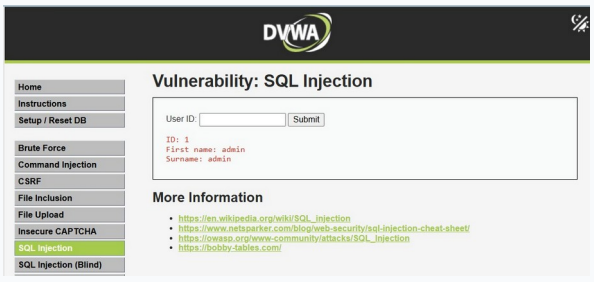

Figure 4A. Normal ID 1 returns one record.

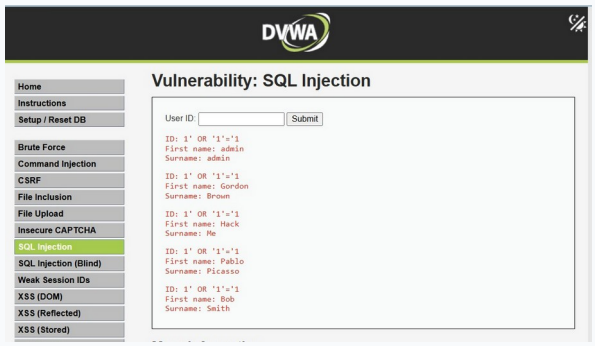

Figure 4B. Boolean injection returns multiple user records.

5. What This Proves
The application inserted user input directly into the SQL statement. The injected always-true condition altered the query and disclosed records that were not requested.

6. How To Fix It
Use prepared statements with bound parameters, validate the ID as an integer, suppress detailed database errors, use a least-privileged database account, and test the application for injection flaws.

LAB 5 | Cross-Site Request Forgery (CSRF)

LOW
Day 5

1. Lab Name & Security Level: Cross-Site Request Forgery (CSRF) - Low

2. What I Was Trying To Do: Make an authenticated DVWA browser submit a password-change request from a separate local HTML page without an anti-CSRF token.

3. Steps I Took

1. Stayed logged in to DVWA as admin and kept the security level set to Low.
2. Created csrf_demo.html containing an auto-submitting GET form directed to the DVWA CSRF endpoint.
3. Set password_new and password_conf to Hacked123! and included the Change parameter.
4. Opened the local HTML file in the same browser session so the existing DVWA session cookie was sent automatically.
5. DVWA displayed “Password Changed.”, confirming that the state-changing request was accepted without a CSRF token.
6. Verified the changed credential and then restored the original lab password after collecting the screenshot.

Payload / Input Used

```
Forged GET request generated by csrf_demo.html:
http://localhost:8001/DVWA/vulnerabilities/csrf/?password_new=Hacked123!
&password_conf=Hacked123!&Change=Change
```

4. Screenshot Evidence

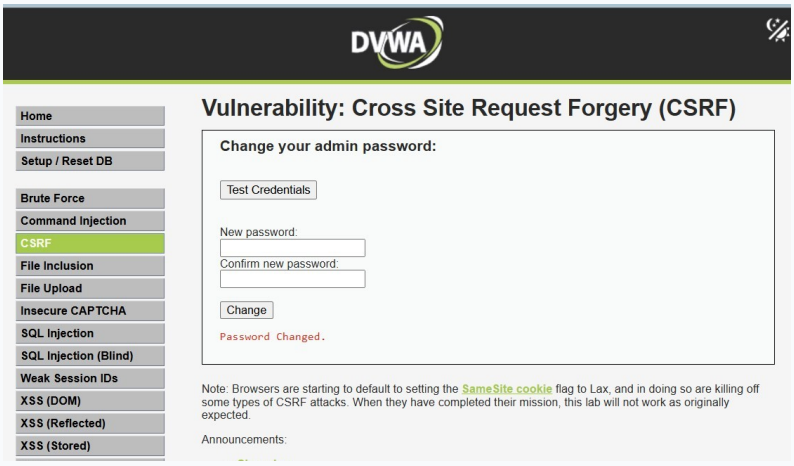

Figure 5. CSRF - DVWA accepts the forged password-change request.

5. What This Proves
The application trusted the authenticated session cookie but did not require a request-specific token. A separate page could therefore trigger a sensitive password change in the victim's browser.

6. How To Fix It
Use unpredictable anti-CSRF tokens validated on the server, require POST for state changes, configure appropriate SameSite cookies, validate Origin/Referer where suitable, and require re-authentication for password changes.

Final Findings and Submission Checklist

The five demonstrations show how weak controls at different layers can be combined into serious security impact. Authentication controls can be defeated through repeated guesses, unsafe command construction can expose the operating system, unrestricted uploads can become code execution, unsafe SQL construction can disclose database records, and missing request-verification controls can let another website trigger sensitive actions.

No.	Requirement	Status
1	Five different DVWA labs documented	**Yes**
2	At least three to four bootcamp days represented	**Yes - Days 2, 3, 4, and 5**
3	Security level stated for every lab	**Yes - Low**
4	Actual payload/input included for every lab	**Yes**
5	What This Proves section completed	**Yes**
6	How To Fix section completed	**Yes**
7	At least one screenshot per lab	Yes - 8 screenshots
8	Local DVWA URL checked	Yes - localhost:8001/DVWA
9	Saved as one PDF or Word file	**Yes**

Evidence note
Eight original screenshots are included: one for Brute Force, two for Command Injection, two for File Upload, two for SQL Injection, and one for CSRF. Together they show baseline behavior where useful and the successful exploit result.

Final document status
The report text, payloads, environment URL, screenshots, findings, fixes, and submission checklist have been completed. The Word and PDF versions were visually checked after export.

Appendix - DevTown Session Links

Day 1: https://www.youtube.com/live/ejcMPXLJVz0

Day 2: https://www.youtube.com/live/avBVW-3tgsQ

Day 3: https://www.youtube.com/live/O1Spur9rEys

Day 4: https://www.youtube.com/live/9NKEQkMk6Vc

Day 5: https://www.youtube.com/live/gPNlDa2dB3Y

> **Ethical-use statement**
>
> The techniques in this document are intended only for DVWA or other systems where the tester has explicit authorization. They must not be used against public websites, third-party accounts, or systems without written permission.